ESPECIFICAÇÃO TÉCNICA · SKILL DO CLAUDE

Gestor Financeiro Inteligente

Sistema web de finanças pessoais e comerciais operado por IA em português: vendas, estoque, empréstimos, clientes, contas, salário PM, Google Agenda e relatórios.

01 Visão geral
02 Arquitetura técnica
03 Módulos e telas
04 Central de Comando com IA
05 Google Agenda
06 Cálculos financeiros
07 Conformidade legal
08 Relatórios em PDF
09 Roteiro de testes
10 Instalação e publicação

Emitido em 12/11/2026 · Banco e cálculos validados em teste (25 fluxos SQL, 11 testes de cálculo)

01

Visão geral

Gerenciador financeiro pessoal e comercial. O diferencial é a **Central de Comando com IA**: o usuário escreve em português ("Venda TZ para Valverde, 71 99401-4023, paga dia 14/12, R$ 150") e a IA cadastra, consulta e atualiza tudo no banco, sincronizando cada vencimento com o Google Agenda.

Regra de ouro: nada é simulado. Não diga que uma função está pronta enquanto ela não funcionar de verdade: dados salvos no banco, evento confirmado pela API do Google, PDF gerado. Se faltar uma credencial (chave da Anthropic, OAuth do Google, projeto Supabase), monte o fluxo de conexão e peça autorização ao usuário. Nunca invente valores de chaves.

Arquivos desta skill (leia antes de cada fase)

Arquivo	Quando ler
`references/arquitetura.md`	Fase 1: stack, pastas, variáveis de ambiente, segurança, LGPD, backup
`assets/schema.sql`	Fase 2: banco completo com RLS, auditoria, exclusão segura, idempotência. Aplique como está e só depois adapte
`references/modulos-e-telas.md`	Fase 3: campos, telas, filtros e indicadores de cada módulo
`references/ia-central-de-comando.md`	Fase 4: prompt de sistema, ferramentas (tool use), fluxos de exemplo, formato da confirmação
`references/google-agenda.md`	Fase 5: OAuth 2.0, fila de sincronização, formato dos eventos, lembretes, recuperação de falhas
`references/calculos-financeiros.md`	Juros, pagamentos parciais, renegociação, KPIs (faturamento ≠ lucro ≠ recebimento ≠ saldo)
`scripts/calculos.mjs`	Implementação de referência dos cálculos, testada. Copie para o projeto em vez de reescrever
`references/conformidade-legal.md`	Antes de criar os módulos de empréstimo e de medicamentos: usura, ANVISA, LGPD
`references/relatorios-pdf.md`	Fase 7: os 13 relatórios e o layout dos PDFs
`references/testes.md`	Fases 8 e 9: roteiro de testes com o resultado esperado no banco e na agenda
`references/instalacao-publicacao.md`	Fase 10: o passo a passo que você entrega ao usuário

Plano de execução (siga na ordem)

1. **Arquitetura.** Confirme a stack recomendada (React + Vite + TypeScript + Tailwind/shadcn no front; Supabase: Postgres, Auth, Edge Functions, Vault; Claude API na Edge Function; Google Calendar API). Se o ambiente do usuário já tiver outra (por exemplo Lovable Cloud, que é Supabase), adapte sem perder nenhuma regra.
2. **Banco.** Aplique `assets/schema.sql`. Confira que todas as tabelas têm RLS ativo e que um usuário não enxerga dados de outro.
3. **Módulos.** Clientes → Produtos/Estoque → Compras → Vendas → Empréstimos → Contas a pagar e receber → Finanças pessoais → Salário PM. Cada módulo grava pelas mesmas funções de serviço que a IA usa (uma única porta de escrita).
4. **Central de Comando com IA.** Edge Function `ia-comando` com tool use. A IA só escreve no banco por ferramentas; a resposta final é montada a partir do retorno real das ferramentas.
5. **Google Agenda.** OAuth com `access_type=offline`, tokens criptografados no banco, fila `agenda_sync_fila` processada por Edge Function agendada (cron) e também logo após cada operação.
6. **Dashboards** com filtros de mês, trimestre, semestre, ano e período personalizado.
7. **PDFs** (`references/relatorios-pdf.md`).
8. **Testes de fluxo** (`references/testes.md`): venda, compra, empréstimo, pagamento total e parcial, despesa, duplicidade.
9. **Teste real da agenda** com a conta do usuário: criar, alterar vencimento, pagar e ver o evento mudar no Google Agenda.
10. **Entrega:** instruções de instalação e publicação, mais uma lista honesta do que foi testado e do que depende de ação do usuário.

As 14 regras do sistema (não negociáveis)

1. Toda operação é salva no banco. A IA só confirma depois que o banco retornou sucesso.
2. Nada some ao recarregar a página: o estado vem do banco, nunca só da memória do navegador.
3. Um cliente = um cadastro. Busque por telefone normalizado (só dígitos, com DDD) e depois por nome aproximado (`pg_trgm`). Se houver mais de um candidato, pergunte.
4. Cada venda fica ligada ao cliente e aos produtos (`venda_itens`).
5. Cada empréstimo tem histórico próprio (`emprestimo_eventos` + `pagamentos`).
6. Toda obrigação com vencimento gera um evento no Google Agenda (via `obrigacoes` → `agenda_eventos`).
7. Mudar o vencimento atualiza o mesmo evento (pelo `google_event_id` salvo), sem criar outro.
8. Pagamentos atualizam saldos automaticamente (trigger `aplicar_pagamento`).
9. Pessoal e comercial não se misturam: a coluna `escopo` ('pessoal' | 'comercial') é obrigatória em obrigações e lançamentos, e os relatórios filtram por ela.
10. Faturamento (vendido), recebimento (entrou no caixa), lucro (faturamento − custo) e saldo disponível (caixa) são indicadores diferentes, com fórmulas em `references/calculos-financeiros.md`.
11. Sem duplicidade: chave de idempotência por comando da IA e verificação de operação igual (mesmo cliente, valor e data) nos últimos 10 minutos, com pergunta antes de gravar.
12. Toda ação da IA termina com uma confirmação que mostra o que foi gravado, os IDs e a situação da agenda.
13. Exclusão é sempre lógica (`excluido_em`). A exclusão definitiva exige uma segunda confirmação digitada, e tudo vai para a `auditoria`.
14. Taxas de juros e vendas de medicamentos passam pelas checagens de `references/conformidade-legal.md`. O sistema registra e avisa; não presume que algo é legal.

Como a IA deve se comportar (resumo; detalhes em `ia-central-de-comando.md`)

- Interpreta datas em relação a hoje, no fuso `America/Bahia`: "dia 14 de dezembro" é o próximo 14/12 que ainda não passou; "em 30 dias" é hoje + 30.

- Pergunta **só** o que falta para gravar. Exemplo: um empréstimo sem modalidade de juros vira a pergunta "Juros simples ou composto? Os 10% são ao mês ou no período todo?". Nunca chuta.
- Nunca inventa telefone, CPF, preço ou custo. Se o produto não tem custo cadastrado, grava a venda e avisa que o lucro fica pendente até o custo ser informado.
- Formato da confirmação:

```
✅ Venda registrada (#V-000123)
Cliente: Valverde (novo cadastro, tel. (71) 99401-4023)
Produto: TZ × 1, R$ 150,00
A receber: R$ 150,00, vencimento 14/12/2026 (#OB-000231)
Google Agenda: ✅ evento criado "COBRAR VALVERDE - R$ 150 - VENDA TZ"
   (ou ⏳ pendente: Google desconectado, vai sincronizar ao reconectar)
```

Verificação antes de dizer que terminou

- Rode `node scripts/calculos.test.mjs` (os cálculos precisam passar).
- Rode o roteiro de `references/testes.md` contra o banco real e mostre as linhas gravadas.
- Mostre o evento criado no Google Agenda (ID e link `htmlLink` retornados pela API).
- Liste com franqueza: ✅ testado e funcionando / ⏳ depende do usuário (credenciais, domínio) / ❌ não feito.

02

Arquitetura técnica

Stack recomendada

Camada	Tecnologia	Por quê
Front-end	React 19 + Vite + TypeScript, TanStack Router e Query, Tailwind 4 + shadcn/ui, Recharts, lucide-react, sonner	Responsivo, rápido, com ótimo suporte a PWA (instala no celular)
Banco	Supabase Postgres com RLS em todas as tabelas, mais `pg_trgm`, `pgcrypto`, `pg_cron` e `pg_net`	Persistência real, segurança por usuário, busca aproximada de nomes, tarefas agendadas
Autenticação	Supabase Auth (e-mail e senha, com 2FA TOTP opcional)	Sessão segura e recuperação de senha
Back-end	Supabase Edge Functions (Deno)	Onde ficam os segredos: chave da Anthropic, client secret do Google, chave de criptografia
IA	Claude API (Messages, com tool use). Modelo padrão `claude-sonnet-5-5`; para frases difíceis, `claude-opus-5-5`	Bom em português e em chamadas de ferramenta
Agenda	Google Calendar API v3 com OAuth 2.0 (fluxo web, servidor)	Integração oficial
PDF	`pdfmake` no navegador (tabelas e totais) + gráficos Recharts exportados como SVG ou PNG	Sem servidor extra
Hospedagem	Front em Vercel, Netlify, GitHub Pages ou Lovable; back no Supabase	Gratuito ou barato para começar

Se o usuário já usa o Lovable, o "Lovable Cloud" é Supabase: o mesmo schema e as mesmas Edge Functions servem.

Estrutura de pastas

```
gestor-financeiro/
├─ src/
│  ├─ routes/           # telas: /, /chat, /clientes, /clientes/$id, /estoque, /compras, /vendas,
│  │                    # /emprestimos, /contas, /pessoal, /salario, /comparativo, /relatorios,
│  │                    # /configuracoes, /entrar
│  ├─ components/       # AppShell (menu lateral + chat flutuante), cards, tabelas, gráficos
│  ├─ lib/
│  │  ├─ supabase.ts    # cliente com chave publicável
│  │  ├─ calculos.ts    # cópia de scripts/calculos.mjs (fonte única dos cálculos)
│  │  ├─ formatos.ts    # BRL, datas pt-BR, telefone, CPF
│  │  └─ pdf/           # um gerador por relatório
├─ supabase/
│  ├─ migrations/0001_schema.sql   # = assets/schema.sql
│  └─ functions/
│     ├─ ia-comando/       # chat + tool use
│     ├─ google-oauth/     # /iniciar e /callback
│     ├─ agenda-sync/      # processa a fila (cron a cada 5 min + chamada imediata)
│     └─ backup/           # exporta JSON criptografado para o Storage (cron diário)
└─ .env.example
```

Uma única porta de escrita

Toda gravação passa por **funções SQL (RPC)** `security invoker` definidas no schema (`registrar_venda`, `registrar_compra`, `registrar_emprestimo`, `registrar_pagamento`, `registrar_despesa`, `alterar_vencimento`, `upsert_cliente`...). As telas e as ferramentas da IA chamam as mesmas funções. Com isso, as regras (estoque, saldos, agenda, auditoria, duplicidade) valem igual para os dois caminhos.

Cada função roda dentro de uma transação: ou grava tudo (venda + itens + baixa de estoque + obrigação + item na fila da agenda), ou não grava nada.

Variáveis de ambiente e segredos

Front (`.env`, público por natureza; quem protege os dados é o RLS):

```
VITE_SUPABASE_URL=
VITE_SUPABASE_PUBLISHABLE_KEY=
```

Segredos das Edge Functions (`supabase secrets set ...`; **nunca** no código nem no git):

```
ANTHROPIC_API_KEY=
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_REDIRECT_URI=https://<projeto>.supabase.co/functions/v1/google-oauth/callback
APP_URL=https://<domínio do app>
TOKEN_ENCRYPTION_KEY=   # 32 bytes em base64, gerado com: openssl rand -base64 32
TZ_PADRAO=America/Bahia
```

Segurança

- **RLS** em todas as tabelas: `user_id = auth.uid()`. Ninguém lê dados de outro usuário, nem com a chave publicável.
- **Criptografia:** os tokens do Google (refresh e access) são cifrados com AES-256-GCM na Edge Function, com a chave `TOKEN_ENCRYPTION_KEY`; o banco guarda só o texto cifrado e o IV. CPF e endereço ficam cifrados com `pgp_sym_encrypt`, com a chave guardada no Supabase Vault; para buscar, usa-se

o hash do CPF.

- **Permissões:** a tabela `membros` tem os papéis `dono`, `operador` (lança, mas não exclui nem exporta) e `leitura`. Isso já prepara o sistema para um contador ou sócio no futuro.
- **Proteção contra exclusão acidental:** exclusão lógica (`excluido_em`), lixeira com restauração em 30 dias e exclusão definitiva só para o `dono`, digitando "EXCLUIR".
- **Histórico de alterações:** o trigger `auditar()` grava antes e depois (JSON) de cada INSERT, UPDATE e DELETE na tabela `auditoria`.
- **Logs de operação:** `ia_operacoes` (cada comando da IA, com as ferramentas chamadas e o resultado) e `agenda_sync_fila` (cada tentativa, com o erro).
- **Tratamento de erros:** toda Edge Function devolve `{ ok, erro, codigo }`; o front mostra um aviso (toast) e nunca diz "salvo" quando houve erro.
- **Backup:** `pg_cron` diário chama a função `backup`, que exporta as tabelas do usuário em JSON cifrado para um bucket privado do Storage e guarda 30 versões. Em Configurações há "Exportar meus dados" (JSON e CSV) e "Restaurar".

LGPD

- Base legal: execução de contrato e legítimo interesse do titular da conta. Os clientes cadastrados são terceiros: registre só o necessário (nome e telefone; CPF e endereço são opcionais e ficam cifrados).
- Tenha uma tela de Privacidade explicando quais dados são guardados, por quê e por quanto tempo.
- Direitos do titular: exportar e excluir os dados de um cliente (anonimização mantendo os valores contábeis).
- Nada de dados pessoais em logs de erro ou em serviços de terceiros, exceto o necessário: Anthropic para interpretar a mensagem e Google para o evento. Avise o usuário disso na tela de Configurações.

Preparado para expansão

- A tabela `contas_financeiras` (carteira, banco, cartão) existe desde o início; todo lançamento aponta para uma conta. Integrações bancárias (Open Finance, extrato OFX) entram depois, gravando em `lancamentos` com `origem='importacao'`.
- `integracoes` é genérica (provedor, tokens cifrados, status). O Google é o primeiro provedor; WhatsApp ou um banco podem entrar depois.

03

Módulos e telas

Layout geral (AppShell)

- **Menu lateral** (no celular, vira uma barra inferior com 5 atalhos e um botão "Mais"): Central IA · Dashboard · Contas · Clientes · Vendas · Compras · Estoque · Empréstimos · Pessoal · Salário PM · Comparativo · Relatórios · Configurações.
- **Barra superior:** seletor de período (Mês | Trimestre | Semestre | Ano | Personalizado, guardado na URL), indicador da agenda, alternância Pessoal / Comercial / Tudo e avatar.
- **Chat flutuante** em todas as telas (botão no canto inferior direito, atalho `Ctrl+K`).
- **Visual de software financeiro premium:** fundo neutro (slate-50 no claro e slate-950 no escuro), cartões brancos com borda sutil, uma cor de destaque discreta (azul-petróleo `#0F766E`), verde para entradas, vermelho para atrasos, números tabulares (`font-variant-numeric: tabular-nums`) e fonte Inter. Inclua tema escuro.
- **Tabelas:** busca, filtros em "chips", ordenação, paginação, exportação CSV e, no celular, linhas que viram cartões.
- **Estados vazios** com um exemplo de comando para a IA ("Experimente: *Vendi 2 TZ para Maria por 300, paga dia 10*").
- O formulário de cada módulo grava pela **mesma RPC** que a IA usa.

Dashboard (`/`)

Cartões: Patrimônio registrado · Saldo disponível (pessoal | comercial) · A receber (atrasado em vermelho) · A pagar · Total emprestado / saldo devedor · Lucro com juros · Faturamento · Lucro das vendas · Despesas pessoais. Gráficos (Recharts): receitas × despesas (barras por mês) · evolução do faturamento e do lucro (linhas) · emprestado × recebido · a pagar × a receber nos próximos 90 dias (fluxo de caixa futuro, em barras empilhadas por semana) · despesas por categoria (rosca) · evolução do valor do estoque · top 10 produtos · top 10 clientes. Listas: vencimentos dos próximos 7 dias · atrasados · alertas de estoque e validade.

Clientes (`/clientes`, `/clientes/$id`)

Lista com busca por nome ou telefone, mostrando total comprado, pendente e atrasado (`v_clientes_resumo`). Ficha: dados (o CPF aparece mascarado, com o botão "mostrar" registrado na auditoria), observações, data do cadastro, e as abas Resumo · Compras · Empréstimos · Pagamentos · Pendências · Linha do tempo. Botões: "Nova venda", "Novo empréstimo", "Registrar recebimento", "Relatório PDF do cliente" e "Mesclar com..." (junta cadastros duplicados, migrando todas as relações e registrando na auditoria).

Estoque e produtos (`/estoque`)

Campos: nome, nome comercial, princípio ativo, apresentação, código ou EAN, unidade, fornecedor, preço de compra (último custo), preço de venda, margem (calculada), estoque atual (só leitura, vem das movimentações), estoque mínimo, classe regulatória, registro ANVISA e controle de lote. Abas: Inventário (em tempo real, via Realtime) · Lotes e validades · Movimentações · Alertas · Mais vendidos · Mais lucrativos. Ajuste de inventário só com motivo (`movimentacoes_estoque.origem='inventario'`).

Compras (`/compras`)

Produto(s), quantidade, custo unitário, total, fornecedor, data, forma de pagamento, vencimento, número de parcelas, lote e validade (obrigatórios para medicamento) e situação do pagamento.

Vendas (`/vendas`)

Cliente, itens (produto, quantidade, preço unitário, desconto), desconto geral, total, custo da mercadoria, lucro bruto, margem %, data, forma de pagamento, data prevista de recebimento, situação (pago, pendente, parcial ou atrasado) e receita médica quando exigida.

Empréstimos (`/emprestimos`)

Lista por situação (ativo, atrasado, quitado, renegociado) e agrupada por cliente. Formulário: cliente, valor, data, taxa, periodicidade, modalidade, prazo (n), número de parcelas e vencimento. O **simulador ao vivo** mostra juros, total, parcelas e a taxa mensal equivalente, com o aviso legal quando aplicável. Detalhe: condições contratuais, parcelas com saldo, pagamentos (juros × principal), linha do tempo (`emprestimo_eventos`: criação, pagamentos, atrasos, prorrogações, renegociações) e os botões Registrar pagamento · Prorrogar · Renegociar · PDF das condições. Relatórios: lucro obtido com juros e valores a receber.

Contas a pagar e a receber (`/contas`)

Abas: A receber de vendas · A receber de empréstimos · A pagar a fornecedores · Despesas pessoais · Parcelamentos. Filtros: vencimento (período), cliente ou fornecedor, categoria, situação e escopo. Ações por linha: pagamento parcial, quitação, alterar vencimento e estornar. Mostra o status da agenda e o link do evento. **Fluxo de caixa futuro:** saldo atual + a receber − a pagar, dia a dia, nos próximos 30, 60 e 90 dias, com a linha do saldo projetado.

Finanças pessoais (`/pessoal`)

Categorias: despesas domésticas, alimentação, combustível, energia, água, internet, escola, saúde, cartões de crédito, financiamentos, empréstimos pessoais, assinaturas e outras. Receitas e despesas **recorrentes** (`recorrencias`, geradas pela rotina diária com 35 dias de antecedência e sincronizadas com a agenda). Indicadores: total de receitas, total de despesas, saldo mensal, contas pagas, pendentes e atrasadas, evolução mês a mês e gastos por categoria. Nunca mostra valores comerciais.

Salário PM (`/salario`)

Lançamento do contracheque por competência: rubricas de provento (salário-base, gratificações, adicionais, horas extras, serviço extraordinário, auxílios, outros) e de desconto (previdência, IR, consignados, plano de saúde, associação, pensão, outros). Gráficos: bruto × líquido × descontos por mês, variação mensal % e evolução anual. Tabela comparativa por rubrica, ano contra ano. A IA entende: "Recebi o salário de novembro: base 5.200, gratificação 1.300, desconto previdência 650, consignado 400".

Comparativo (`/comparativo`)

Mês atual × mês anterior (ou × mesmo mês do ano anterior): cartões de variação de faturamento, número de vendas, itens vendidos, lucro bruto, margem média, ticket médio, clientes novos e recorrentes. Gráfico de 12 meses, ranking de produtos e clientes, e o texto "Crescimento de 30% no faturamento; lucro cresceu 22%". Faturamento e lucro aparecem sempre separados.

Configurações (`/configuracoes`)

Perfil e 2FA · Contas financeiras · Categorias · Agenda (conectar, lembretes, horário, ação ao pagar, sincronizar ou reconstruir) · Licenças sanitárias · Limite legal de juros · Membros e permissões · Backup e exportação · Privacidade (LGPD) · Lixeira (restaurar).

04

Central de Comando com IA

A tela principal é um chat. Também há um botão flutuante, presente em todas as telas, que abre o mesmo chat em um painel lateral (no celular, em tela cheia).

Como funciona

```
Navegador ──(JWT do usuário)──► Edge Function ia-comando ──► Claude API (tool use)
                                  │  ◄── tool_use: registrar_venda {...}
                                  ├──► supabase.rpc('registrar_venda', p)   ← com o JWT do usuário: o RLS vale
                                  │  ──► tool_result: {ok:true, venda_id...} ou {erro:'ESTOQUE_INSUFICIENTE...'}
                                  ├──► dispara agenda-sync para as obrigações criadas e espera até 8 s pelo resultado
                                  └──► resposta final + grava chat_mensagens e ia_operacoes
```

- A Edge Function cria o cliente Supabase **com o token do usuário** (`Authorization` repassado), nunca com a service_role. Assim a IA só enxerga e grava os dados do próprio usuário.
- Cada mensagem recebe uma `chave_idempotencia` = `ia:<conversa_id>:<id da mensagem>:<índice da ferramenta>`. Se a rede cair e o usuário reenviar, nada é gravado em dobro.
- Loop de ferramentas: no máximo 8 voltas por mensagem. Se o limite estourar, responda dizendo o que foi e o que não foi gravado.
- Histórico: envie as últimas 20 mensagens da conversa e um bloco "Contexto de hoje" (data, fuso, se o Google está conectado, contas e categorias existentes).

Prompt de sistema (use como está e preencha as variáveis)

```
Você é o assistente do GESTOR FINANCEIRO INTELIGENTE de {{nome_usuario}}. Hoje é {{hoje_extenso}} ({{hoje_iso}}), fuso {{fuso}}.
Você registra e consulta movimentações financeiras SOMENTE por meio das ferramentas. Fale português do Brasil, de forma curta e objetiva.

REGRAS
1. Nunca diga que algo foi salvo sem uma ferramenta ter retornado ok:true. Se uma ferramenta retornar erro, explique em linguagem simples e diga o que falta.
2. Nunca invente dados (telefone, CPF, preço, custo, data, taxa). Se faltar algo OBRIGATÓRIO, pergunte só isso, numa única pergunta.
   Obrigatórios: venda = cliente, produto, valor (ou quantidade e preço), vencimento ou "pago agora".
   Empréstimo = cliente, valor, taxa, se a taxa é ao mês/dia/ano ou no período todo, prazo ou vencimento, juros simples ou composto (quando houver mais de um período).
   Compra = produto, quantidade, custo unitário ou total, vencimento ou "pago agora". Despesa = descrição, valor, se já foi paga ou o vencimento.
3. Clientes: SEMPRE chame buscar_cliente antes de criar. Se vier telefone, ele manda. Se houver candidatos ambíguos, pergunte qual é.
   Crie um cliente novo só quando a busca não encontrar ninguém e diga na confirmação que é um "novo cadastro".
4. Produtos: chame buscar_produto. Se não existir, crie com cadastrar_produto (só com os dados ditos) e avise que faltam custo/validade, quando for o caso.
   Se o nome sugerir medicamento (princípio ativo, mg, ampola, caneta...), pergunte a classe regulatória e o registro ANVISA antes de cadastrar.
5. Datas: interprete em relação a hoje. "dia 20" = o próximo dia 20 que ainda não passou; "14 de dezembro" = o próximo 14/12; "em 30 dias" = hoje + 30.
   Se a data for vaga ("quando puder", "fim do mês que vem?"), pergunte.
6. Pessoal x comercial: venda, compra, empréstimo e estoque são COMERCIAIS. "Minha casa", contas da família, salário PM são PESSOAIS. Na dúvida, pergunte.
7. Faturamento, recebimento, lucro e saldo são coisas diferentes. Use exatamente o que as ferramentas de consulta retornarem.
8. Juros: se a ferramenta retornar AVISO_LEGAL_PENDENTE, mostre o aviso (taxa equivalente x referência legal) e pergunte se o usuário quer registrar
   mesmo assim, ciente. Só então chame de novo com ciente_aviso_legal=true. Nunca afirme que a taxa é legal.
9. Medicamentos: se a ferramenta bloquear (LICENCA_OBRIGATORIA, RECEITA_OBRIGATORIA, CONTROLADO_BLOQUEADO, REGISTRO_ANVISA_AUSENTE),
   explique o motivo e o que precisa ser cadastrado. Não ofereça meios de contornar.
10. POSSIVEL_DUPLICIDADE: mostre a operação parecida que já existe e pergunte se é outra. Só repita com confirmar_duplicada=true se o usuário confirmar.
11. Pagamento recebido: localize a obrigação com listar_contas (cliente + valor + descrição). Se houver mais de uma em aberto que sirva, pergunte qual.
    Se o valor for menor que o saldo, é pagamento PARCIAL: registre e informe o saldo restante e o próximo vencimento.
12. Toda resposta que gravou algo termina com o bloco de confirmação (modelo abaixo), mostrando os números (#V-, #E-, #OB-) retornados.
    A linha "Google Agenda" usa SOMENTE o status retornado por status_agenda: ✅ sincronizado, ⏳ pendente (com o motivo) ou ❌ erro.

MODELO DE CONFIRMAÇÃO
✅ <Operação> registrada (#<número>)
Cliente/Fornecedor: <nome> (<novo cadastro | cadastro existente>, tel. <telefone, se houver>)
<linhas com produto, quantidade, valores, juros, lucro, conforme o caso>
<A receber | A pagar>: R$ <valor>, vencimento <dd/mm/aaaa> (#OB-<número>)
Google Agenda: <status>
```

Ferramentas (definições para a Claude API)

Cada ferramenta corresponde a uma RPC do schema ou a uma consulta. Todas recebem e devolvem JSON. Abaixo, `input_schema` resumido (escreva o JSON Schema completo no código).

Ferramenta	Faz	Entrada principal
`buscar_cliente`	`rpc('buscar_clientes', {p_texto})`	`texto` (nome ou telefone)
`cadastrar_ou_atualizar_cliente`	`rpc('upsert_cliente')`; se vier CPF ou endereço, a Edge Function cifra antes de gravar	`nome`, `telefone?`, `cpf?`, `endereco?`, `observacoes?`

Ferramenta	Faz	Entrada principal
`buscar_produto`	`select` em `produtos` por `nome_busca % texto` ou código	`texto`
`cadastrar_produto`	`insert` em `produtos`	`nome`, `preco_venda?`, `preco_compra?`, `unidade?`, `classe_regulatoria?`, `registro_anvisa?`, `principio_ativo?`, `apresentacao?`, `estoque_minimo?`
`registrar_venda`	`rpc('registrar_venda', {p})`	`cliente_id`, `itens[{produto_id, quantidade, preco_unitario, desconto?}]`, `vencimento?`, `pago_na_hora?`, `forma_pagamento?`, `desconto?`, `receita_medica_id?`, `confirmar_duplicada?`
`registrar_compra`	`rpc('registrar_compra', {p})`	`fornecedor_nome?` ou `fornecedor_id?`, `itens[{produto_id, quantidade, custo_unitario, lote?, validade?}]`, `vencimento?`, `parcelas?`, `pago_na_hora?`
`registrar_emprestimo`	`rpc('registrar_emprestimo', {p})`	`cliente_id`, `principal`, `taxa_pct`, `periodicidade`, `modalidade`, `periodos`, `parcelas?`, `vencimento`, `ciente_aviso_legal?`
`registrar_pagamento`	`rpc('registrar_pagamento')`	`obrigacao_id`, `valor`, `data?`, `forma?`
`registrar_conta`	`rpc('registrar_conta')`: despesa (`tipo:'pagar'`) ou receita avulsa (`'receber'`)	`descricao`, `valor`, `escopo`, `categoria_slug`, `pago?`, `vencimento?`, `parcelas?`
`criar_recorrencia`	`insert` em `recorrencias`	`descricao`, `tipo`, `escopo`, `categoria_slug`, `valor`, `dia_vencimento`
`alterar_vencimento`	`rpc('alterar_vencimento')`	`obrigacao_id`, `nova_data`, `motivo?`
`renegociar_emprestimo`	`rpc('renegociar_emprestimo')`	`emprestimo_id`, novas condições
`estornar_pagamento`	`rpc('estornar_pagamento')`: só depois de o usuário confirmar	`pagamento_id`, `motivo`
`registrar_contracheque`	`rpc('registrar_contracheque')`	`competencia`, `data_pagamento`, `itens[]`
`registrar_receita_medica`	`insert` em `receitas_medicas`	`cliente_id`, `prescritor`, `registro_prescritor`, `data_emissao`, `tipo`, `retida`
`listar_contas`	`select` em `v_obrigacoes` com filtros	`tipo?`, `cliente_id?`, `situacao?`, `de?`, `ate?`, `texto?`
`consultar_cliente`	`v_clientes_resumo` + últimas vendas, empréstimos e pagamentos	`cliente_id`
`consultar_emprestimos`	`emprestimos` + `emprestimo_eventos`	`cliente_id?`, `status?`
`consultar_estoque`	`produtos` + `v_estoque_alertas`	`texto?`
`resumo_financeiro`	KPIs do período (ver calculos-financeiros.md)	`de`, `ate`, `escopo?`
`status_agenda`	lê `agenda_eventos` das obrigações informadas, depois de a função ter disparado `agenda-sync`	`obrigacao_ids[]`

Descrições das ferramentas: escreva em português, dizendo **quando usar** e **o que nunca fazer**. Exemplo para `registrar_venda`: "Grava uma venda já confirmada pelo usuário. Antes, obtenha cliente_id com buscar_cliente/cadastrar_ou_atualizar_cliente e produto_id com buscar_produto. Não use para empréstimos."

Os 5 exemplos do usuário: sequência esperada

1. "Venda TZ para Valverde, telefone 71 99401-4023, pagamento será dia 14 de dezembro, valor 150 reais." `buscar_cliente("71 99401-4023")` → vazio → `cadastrar_ou_atualizar_cliente({nome:"Valverde", telefone:"71 99401-4023"})` → `buscar_produto("TZ")` → (se não existir: `cadastrar_produto({nome:"TZ", preco_venda:150})`) → `registrar_venda({cliente_id, itens:[{produto_id, quantidade:1, preco_unitario:150}], vencimento:"2026-12-14"})` → `status_agenda([obrigacao_id])` → confirmação com "COBRAR VALVERDE - R$ 150 - VENDA TZ". Se o TZ não tiver estoque, a RPC devolve `ESTOQUE_INSUFICIENTE`: pergunte se quer lançar a compra antes ou ajustar o estoque.

2. "Emprestei 1.000 reais para João, juros de 10%, pagamento em 30 dias." Faltam a modalidade e a base da taxa. Como é um único período de 30 dias, simples e composto dão o mesmo valor; pergunte só "Os 10% são ao mês ou no período todo?" quando isso mudar o resultado (aqui não muda: os dois dão R$ 1.100). → `buscar_cliente("João")` (se houver vários Joões, pergunte) → `registrar_emprestimo({principal:1000, taxa_pct:10, periodicidade:"mensal", modalidade:"simples", periodos:1, vencimento: hoje+30})` → `AVISO_LEGAL_PENDENTE` → mostre o aviso → com o "sim" do usuário, repita com `ciente_aviso_legal:true` → confirmação: principal R$ 1.000, juros R$ 100, total R$ 1.100, vencimento, "RECEBER JOÃO - R$ 1.100 - EMPRÉSTIMO".

3. "Valverde pagou os 150 reais da venda TZ." `buscar_cliente("Valverde")` → `listar_contas({cliente_id, tipo:"receber", situacao:["pendente","parcial","atrasado"], texto:"TZ"})` → uma obrigação de R$ 150 → `registrar_pagamento({obrigacao_id, valor:150})` → `status_agenda` → "Recebimento registrado. Venda #V-... quitada. Evento marcado como PAGO na agenda."

4. "Comprei 20 unidades de TZ por 80 reais cada, pagamento para dia 20." `buscar_produto("TZ")` → `registrar_compra({itens:[{produto_id, quantidade:20, custo_unitario:80}], vencimento:"2026-11-20"})` → confirmação: estoque passou de X para X+20, custo unitário R$ 80, total R$ 1.600, a pagar em 20/11, "PAGAR FORNECEDOR - R$ 1.600". O fornecedor não foi dito: grave sem fornecedor e pergunte depois, de forma opcional ("Quer informar o fornecedor?"). Para medicamento, pergunte o lote e a validade (são obrigatórios para a rastreabilidade).

5. "Paguei 350 reais de energia da minha casa." `registrar_conta({tipo:"pagar", descricao:"Energia - casa", valor:350, escopo:"pessoal", categoria_slug:"energia", pago:true})` → confirmação: despesa pessoal, categoria Energia, paga hoje, saldo pessoal atualizado. Não há evento na agenda porque não ficou nada pendente.

Esqueleto da Edge Function `ia-comando` (Deno)

```
import { createClient } from 'npm:@supabase/supabase-js@2';
import Anthropic from 'npm:@anthropic-ai/sdk';
import { TOOLS, executarFerramenta } from './ferramentas.ts';   // definições + switch que chama as RPCs
```

```
import { montarSystemPrompt } from './prompt.ts';

const anthropic = new Anthropic({ apiKey: Deno.env.get('ANTHROPIC_API_KEY')! });

Deno.serve(async (req) => {
  if (req.method === 'OPTIONS') return new Response(null, { headers: cors });
  const auth = req.headers.get('Authorization');
  if (!auth) return json({ ok: false, erro: 'nao_autenticado' }, 401);
  const db = createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_ANON_KEY')!, {
    global: { headers: { Authorization: auth } },              // RLS do próprio usuário
  });
  const { data: { user } } = await db.auth.getUser();
  if (!user) return json({ ok: false, erro: 'sessao_invalida' }, 401);

  const { conversa_id, mensagem_id, texto } = await req.json();
  const { data: op } = await db.from('ia_operacoes').insert({ conversa_id, mensagem: texto }).select().single();
  await db.from('chat_mensagens').insert({ conversa_id, papel: 'user', conteudo: { texto } });

  const historico = await carregarHistorico(db, conversa_id, 20);    // [{role, content}]
  const messages = [...historico, { role: 'user', content: texto }];
  const log: unknown[] = [];
  const obrigacoesTocadas = new Set<string>();

  try {
    for (let volta = 0; volta < 8; volta++) {
      const resp = await anthropic.messages.create({
        model: Deno.env.get('IA_MODELO') ?? 'claude-sonnet-5-5',
        max_tokens: 2048,
        system: await montarSystemPrompt(db, user),
        tools: TOOLS,
        messages,
      });
      messages.push({ role: 'assistant', content: resp.content });
      if (resp.stop_reason !== 'tool_use') {
        const resposta = resp.content.filter((b) => b.type === 'text').map((b) => b.text).join('\n');
        await db.from('chat_mensagens').insert({ conversa_id, papel: 'assistant', conteudo: { texto: resposta } });
        await db.from('ia_operacoes').update({ ferramentas: log, resposta, status: 'concluida' }).eq('id', op.id);
        return json({ ok: true, resposta });
      }
      const resultados = [];
      for (const [i, bloco] of resp.content.filter((b) => b.type === 'tool_use').entries()) {
        const chave = `ia:${conversa_id}:${mensagem_id}:${volta}:${i}`;
        const t0 = Date.now();
        const saida = await executarFerramenta(db, bloco.name, bloco.input, chave, obrigacoesTocadas);
        log.push({ nome: bloco.name, entrada: bloco.input, saida, ms: Date.now() - t0 });
        resultados.push({ type: 'tool_result', tool_use_id: bloco.id, content: JSON.stringify(saida), is_error: !saida.ok });
      }
      messages.push({ role: 'user', content: resultados });
    }
    throw new Error('limite_de_voltas');
  } catch (e) {
    await db.from('ia_operacoes').update({ ferramentas: log, status: 'erro', erro: String(e) }).eq('id', op.id);
    return json({ ok: false, erro: 'Não consegui concluir. Nada além do que aparece no histórico foi gravado.', detalhe: String(e) }, 500);
  }
});
```

Em `executarFerramenta`: - Capture o erro da RPC e devolva `{ ok:false, codigo, mensagem }`, com o `codigo` sendo o prefixo antes de ":" na mensagem (`ESTOQUE_INSUFICIENTE`, `AVISO_LEGAL_PENDENTE`, ...). - Em `status_agenda`, chame `agenda-sync` com `{ obrigacao_ids }` e espere até 8 s. Depois leia `agenda_eventos`. Se o Google estiver desconectado, devolva `pendente` com o motivo `google_desconectado`.

Tela do chat

- Bolhas, com a confirmação mostrada como um cartão (ícone, números e links para o registro).
- Os "chips" de status da agenda atualizam sozinhos (Supabase Realtime em `agenda_eventos`).
- Sugestões rápidas: "Quem está me devendo?", "Resumo do mês", "Estoque baixo", "Vencimentos desta semana".
- Entrada por voz (Web Speech API, `pt-BR`) no celular.
- Botão "Desfazer" nas confirmações dos últimos 10 minutos: chama `estornar_pagamento` ou faz o cancelamento lógico, sempre com uma pergunta de confirmação antes.

05

Google Agenda

Regra: toda obrigação com vencimento (`obrigacoes` com status `pendente` ou `parcial`) tem um evento no Google Agenda. O banco garante isso com o trigger `obrigacoes_agenda`, que coloca a obrigação na fila; a Edge Function `agenda-sync` cria, atualiza ou cancela o evento. A sincronização só conta como feita quando a API do Google responde 200 e o `google_event_id` é salvo.

1. Configuração no Google Cloud (o dono do app faz uma vez)

1. Acesse console.cloud.google.com, crie o projeto "Gestor Financeiro" e ative a **Google Calendar API**.
2. Em "Tela de consentimento OAuth", escolha o tipo **Externo**, informe o nome do app e o e-mail de suporte, e adicione o escopo `https://www.googleapis.com/auth/calendar.events`. Inclua a conta Google do usuário em "Usuários de teste" (enquanto o app não for verificado pelo Google).
3. Em "Credenciais", crie um **ID do cliente OAuth** do tipo "Aplicativo da Web": - Origem JavaScript: a URL do app (`APP_URL`). - URI de redirecionamento: `https://<projeto>.supabase.co/functions/v1/google-oauth/callback`.
4. Grave os segredos no Supabase: `supabase secrets set GOOGLE_CLIENT_ID=... GOOGLE_CLIENT_SECRET=... GOOGLE_REDIRECT_URI=... TOKEN_ENCRYPTION_KEY=$(openssl rand -base64 32)`.

> Em modo "Teste", o Google expira o refresh token em 7 dias. Para uso contínuo, publique o app na tela de consentimento. Para uso pessoal, com menos de 100 usuários, não é preciso passar pela verificação completa, apenas aceitar o aviso de "app não verificado".

2. Fluxo OAuth 2.0 (Edge Function `google-oauth`)

`POST /google-oauth/iniciar` (com o JWT do usuário): 1. Gera um `state` aleatório de 32 bytes e grava em `integracoes.oauth_state`, válido por 10 min (a tabela é acessada via service_role). 2. Devolve a URL `https://accounts.google.com/o/oauth2/v2/auth?client_id=…&redirect_uri=…&response_type=code&scope=https://www.googleapis.com/auth/calendar.events%20openid%20email&access_type=offline&prompt=consent&include_granted_scopes=` 3. O front abre essa URL e o usuário autoriza.

`GET /google-oauth/callback?code&state`: 1. Procura a integração pelo `state`, confere a validade e apaga o `state` (anti-CSRF). 2. `POST https://oauth2.googleapis.com/token` com `grant_type=authorization_code`. 3. Cifra `access_token` e `refresh_token` com AES-256-GCM (`crypto.subtle`, IV aleatório de 12 bytes, formato `base64(iv).base64(cifra)`). 4. Salva `status='conectado'`, `conta_email` (do id_token), `expira_em` e `escopos`. 5. **Reprocessa a fila:** `update agenda_sync_fila set proxima_tentativa = now(), tentativas = 0 where user_id = … and processado_em is null`, e chama `agenda-sync`. 6. Redireciona para `APP_URL/configuracoes?google=conectado`.

Renovação: antes de cada chamada, se `expira_em < now() + 60s`, faça `grant_type=refresh_token`. Se a resposta for `invalid_grant`, marque `status='revogado'`, mantenha os itens na fila e mostre no app o aviso "Reconectar Google Agenda".

Desconectar: `POST https://oauth2.googleapis.com/revoke?token=…`, apague os tokens e marque `desconectado`. A fila continua guardando as alterações.

3. Processador da fila (Edge Function `agenda-sync`)

Chamado pelo `pg_cron` a cada 5 minutos, logo após cada operação (pela `ia-comando` e pelas telas) e após a reconexão. Usa a service_role; com `{obrigacao_ids}`, processa só essas.

```
para cada item da fila com processado_em is null e proxima_tentativa <= now()  (lote de 50, "for update skip locked"):
  integ = integracoes do user;  se status != 'conectado' → ultimo_erro='google_desconectado', reagenda em 1h, continue
  o = v_obrigacoes(item.obrigacao_id);  ev = agenda_eventos(item.obrigacao_id)
  decidir:
    item.acao='cancelar' ou o excluída/cancelada/renegociada  → DELETE do evento (se existir); ev.status='cancelado'
    o.saldo = 0 (pago):
        sem google_event_id → ev.status='cancelado' (não houve pendência; nada a criar)
        config.agenda_ao_pagar='excluir'     → DELETE; ev.status='cancelado'
        config.agenda_ao_pagar='marcar_pago' → PATCH título "✅ PAGO - <título>", colorId "2" (verde), remove lembretes
    pendente/parcial:
        corpo = montarEvento(o)                      (abaixo)
        hash = sha256(JSON(corpo));  se hash == ev.hash_conteudo e ev.status='sincronizado' → nada a fazer
        se ev.google_event_id → PATCH events/{id}
            404/410 (apagado à mão no Google) → cria de novo (passo abaixo)
        senão → antes de criar, procure um evento órfão:
            GET events?privateExtendedProperty=gfi_obrigacao_id=<o.id>   (evita duplicar se uma resposta anterior se perdeu)
            achou → PATCH nele; não achou → POST events
  sucesso (2xx) → salva google_event_id, html_link, hash, status='sincronizado', ultima_sync=now(); item.processado_em=now()
  erro → tentativas+1; proxima_tentativa = now() + min(2^tentativas, 360) min; ultimo_erro = mensagem; ev.status='erro'
         401 → renova o token e tenta 1 vez;  429/5xx → backoff;  depois de 10 tentativas, alerta no app
```

Por que não há duplicidade: (1) o índice único `agenda_fila_unica` permite um só item pendente por obrigação; (2) `agenda_eventos.obrigacao_id` é único; (3) cada evento leva `extendedProperties.private.gfi_obrigacao_id`, então mesmo que o banco perca o ID, o evento é reencontrado em vez de recriado.

4. Formato do evento

```
{
  "summary": "COBRAR VALVERDE - R$ 150 - VENDA TZ",
  "description": "Cliente: Valverde\nTelefone: (71) 99401-4023\nValor em aberto: R$ 150,00 (total R$ 150,00)\nOperação: Venda #V-000123 · Parcela 1/1\nVencimento: 14/12/2026\nID no sistema: OB-000231 (a9f3…)\nAbrir: https://app/contas/a9f3…",
  "start": { "dateTime": "2026-12-14T09:00:00", "timeZone": "America/Bahia" },
  "end":   { "dateTime": "2026-12-14T09:30:00", "timeZone": "America/Bahia" },
```

```
  "reminders": { "useDefault": false, "overrides": [
      { "method": "popup", "minutes": 4320 }, { "method": "popup", "minutes": 1440 }, { "method": "popup", "minutes": 0 } ] },
  "extendedProperties": { "private": { "gfi_obrigacao_id": "a9f3…", "gfi_origem": "venda", "gfi_versao": "1" } },
  "colorId": "11"
}
```

- Títulos: venda → `COBRAR <CLIENTE> - R$ <valor> - VENDA <PRODUTOS>`; empréstimo → `RECEBER <CLIENTE> - R$ <valor> - EMPRÉSTIMO` (com `(2/3)` quando houver parcelas); compra ou fornecedor → `PAGAR <FORNECEDOR> - R$ <valor>`; despesa pessoal → `PAGAR <DESCRIÇÃO> - R$ <valor>`. Use `tituloEvento()` de `scripts/calculos.mjs`.
- O valor no título é o **saldo em aberto**. Num pagamento parcial, o evento é atualizado com o novo saldo e a descrição ganha "Pago até agora: R$ X".
- Lembretes vêm de `configuracoes.lembretes_minutos` (padrão: 3 dias, 1 dia e no dia), editáveis em Configurações → Agenda. O Google aceita no máximo 5 lembretes, de até 40.320 min (4 semanas). Mudar a configuração coloca todos os eventos pendentes na fila.
- Horário: `configuracoes.hora_evento` (padrão 09:00) no fuso do usuário.
- Cores: a receber em vermelho (11), a pagar em laranja (6), pago em verde (2).

5. Indicador de sincronização

- Na barra superior: o ponto verde "Agenda sincronizada" aparece quando a fila está vazia e o status é `conectado`. "⏳ N pendentes" quando há itens na fila. Vermelho, com "Reconectar", quando o status é `revogado` ou `erro`.
- Em cada linha de conta a pagar ou a receber: um ícone de status (`v_obrigacoes.agenda_status`) e um link para o evento (`agenda_link`).
- Em Configurações → Agenda: conta conectada, últimos erros, botão "Sincronizar agora" (chama `agenda-sync`) e botão "Reconstruir agenda" (coloca todas as obrigações em aberto na fila; não duplica nada, graças ao `gfi_obrigacao_id`).

6. Teste real (obrigatório antes de dizer que está pronto)

1. Conecte a conta Google do usuário.
2. Registre a venda do Valverde e confira no Google Agenda o evento em 14/12, às 09:00, com 3 lembretes. Registre o `htmlLink` no relatório.
3. Altere o vencimento para 15/12: é **o mesmo** evento que muda de dia (o ID não muda).
4. Registre um pagamento parcial de R$ 50: o título passa a ser "COBRAR VALVERDE - R$ 100 - VENDA TZ".
5. Registre o restante: o título vira "✅ PAGO - ..." (ou o evento some, se a configuração for `excluir`).
6. Desconecte o Google, registre uma despesa com vencimento (ela fica ⏳ pendente), reconecte e veja o evento ser criado sozinho.
7. Apague um evento à mão no Google e clique em "Sincronizar agora": ele é recriado, uma vez só.

06

Cálculos financeiros

A implementação de referência está em `scripts/calculos.mjs`, coberta por testes. O banco repete as mesmas fórmulas em `calcular_emprestimo` e `taxa_mensal_equivalente`.

Os quatro conceitos que nunca se confundem

Conceito	Definição	De onde vem
Faturamento	Soma do valor das vendas no período, pela data da venda, recebidas ou não	`sum(vendas.total)`
Recebimento	Dinheiro que de fato entrou no período	`sum(lancamentos.valor)` com `tipo='entrada'`, pela data do pagamento
Lucro bruto de vendas	Faturamento − custo da mercadoria vendida (CMV, pelo custo do lote ou o último custo)	`vendas.lucro_bruto`
Lucro com juros	Só a parte de juros dos pagamentos de empréstimos já recebidos	`sum(pagamentos.juros_valor)`
Saldo disponível	Saldo inicial das contas + entradas − saídas	`v_saldo_contas`

- Venda sem custo cadastrado entra no faturamento, mas **não** no lucro. A tela mostra "N vendas sem custo: lucro incompleto".
- Dinheiro emprestado **não é despesa**, e o principal devolvido **não é receita**: os dois são lançamentos `natureza='patrimonial'`. Só os juros são receita.
- Compra de estoque é saída de caixa quando é paga; vira custo (CMV) quando a mercadoria é vendida.

Indicadores do dashboard (período filtrado)

- **Patrimônio financeiro registrado** = saldo das contas + total a receber em aberto (vendas + empréstimos) + estoque ao custo − total a pagar em aberto.
- **Saldo disponível** = `v_saldo_contas`, separado em pessoal e comercial, mais o total.
- **Total a receber / a pagar** = `sum(saldo)` em `v_obrigacoes` com situação pendente, parcial ou atrasado (mostre o atrasado à parte).
- **Total emprestado** = principal dos empréstimos ativos, atrasados e renegociados; **saldo devedor** = soma dos saldos das obrigações deles.
- **Faturamento comercial, lucro das vendas, margem média e ticket médio**: ver `v_vendas_mensal`.
- **Despesas pessoais** = lançamentos `escopo='pessoal'`, `tipo='saida'`, `natureza='operacional'`.

Empréstimos

- **Simples:** `M = P × (1 + i × n)`. **Composto:** `M = P × (1 + i)^n`, com `i` = taxa por período e `n` = número de períodos.
- **Periodicidade** (`diaria`, `semanal`, `mensal`, `anual` ou `periodo_total`) define a unidade de `n`. "10% em 30 dias" = mensal, n = 1 → R$ 1.100.
- **Parcelas:** valores iguais, com os juros distribuídos de forma proporcional entre elas e a última absorvendo o arredondamento. Se o usuário pedir tabela Price, implemente `PMT = P·i/(1−(1+i)^−n)` à parte, sem trocar o padrão.
- **Pagamento parcial:** abate primeiro os juros da parcela e depois o principal (Código Civil, art. 354). O saldo restante continua com o mesmo vencimento, e o evento da agenda é atualizado com o novo valor.
- **Atraso:** a rotina diária marca o empréstimo como `atrasado` e registra o evento `atraso`. **Não** aplique multa ou juros de mora automaticamente. Encargos de mora só entram se tiverem sido contratados e registrados nas condições, e respeitando os limites legais.
- **Prorrogação:** `alterar_vencimento` (mesmo evento na agenda, histórico em `emprestimo_eventos`).
- **Renegociação:** `renegociar_emprestimo` encerra as parcelas em aberto com status `renegociado` (os eventos são cancelados) e cria novas parcelas sobre o saldo devedor, com novas condições.
- **Situação:** `ativo` → (`atrasado`) → `quitado`; `renegociado` quando houve nova negociação.
- **Relatório de lucro com juros:** `sum(pagamentos.juros_valor)` por mês e por cliente, comparado com os juros previstos.
- **Taxa mensal equivalente:** é calculada sempre e comparada com `configuracoes.limite_juros_mensal_pct` (ver conformidade-legal.md).

Comparativo mensal de vendas

Para o mês M e o mês anterior M−1 (ou o mesmo mês do ano anterior, como opção):

- Crescimento % = `(M − (M−1)) / (M−1) × 100`; quando M−1 = 0, mostre "—" e não "∞". Exemplo: outubro R$ 5.000 e novembro R$ 6.500 = **+30%**.
- Compare: faturamento, quantidade de vendas, quantidade de itens, lucro bruto, margem média, ticket médio, clientes novos (primeira compra no mês) e clientes recorrentes.
- Top produtos por quantidade e por lucro, e top clientes por faturamento.
- Os cartões mostram o valor, a seta e a variação %, sempre com o rótulo "Faturamento" ou "Lucro" explícito.

Estoque

- Entrada pela compra, com o custo do lote e a atualização de `produtos.preco_compra`.
- Saída pela venda, em ordem FEFO (vence primeiro, sai primeiro). Lotes vencidos nunca são vendidos.
- Margem do produto = `(preço de venda − custo) / preço de venda`. Na tela, mostre também o markup = `(venda − custo) / custo`.
- Valor do estoque = `Σ quantidade do lote × custo do lote`.
- Alertas: `estoque_atual <= estoque_minimo`; validade em até 60 dias (o prazo é configurável); lote vencido com saldo (sugira descarte com registro).

07

Conformidade legal

> O sistema **registra, avisa e bloqueia o que é claramente proibido**. Ele não emite parecer jurídico. As referências abaixo servem como ponto de partida e devem ser confirmadas com um advogado ou contador antes do uso comercial, porque leis e normas mudam. Mostre essa ressalva na tela de Configurações → Conformidade.

1. Empréstimo de dinheiro a juros entre particulares

O que considerar (confirme a redação em vigor):

- **Lei de Usura (Decreto 22.626/1933):** limita os juros nos contratos entre particulares e veda o anatocismo (juros sobre juros) em período inferior a um ano (art. 4º; Súmula 121 do STF). A Lei 14.905/2024 afastou a Lei de Usura para alguns casos (por exemplo, obrigações entre pessoas jurídicas e com instituições financeiras), mas **não** para o mútuo entre pessoas físicas.
- **Código Civil:** art. 591 (mútuo com fins econômicos: juros presumidos, limitados à taxa do art. 406 e capitalização só anual); art. 406 (taxa legal, redefinida pela Lei 14.905/2024 como Selic menos IPCA); art. 354 (pagamento abate primeiro os juros).
- **Lei 1.521/1951, art. 4º:** cobrar juros acima da taxa permitida em lei é **crime contra a economia popular** (usura pecuniária).
- **Lei 7.492/1986, art. 16:** operar como instituição financeira sem autorização (por exemplo, emprestar a juros de forma habitual, como atividade) pode configurar crime contra o sistema financeiro.
- **Cobrança:** é proibido expor o devedor ao ridículo ou fazer ameaças (CP art. 147 e art. 345, exercício arbitrário das próprias razões). Os lembretes do sistema são só para o usuário, nunca mensagens automáticas ao devedor.

Como o sistema aplica isso:

1. `configuracoes.limite_juros_mensal_pct` começa em **1% a.m.** (referência conservadora de 12% a.a.). O usuário pode alterar, e a tela avisa que o valor deve ser confirmado com um advogado.
2. Todo empréstimo calcula a **taxa mensal equivalente**. Acima do limite, a RPC devolve `AVISO_LEGAL_PENDENTE` e a IA mostra:

   > "⚠️ A taxa informada equivale a 10% ao mês. Para empréstimo entre pessoas físicas, a legislação limita os juros (Lei de Usura e Código Civil) e a cobrança acima do permitido pode ser crime (Lei 1.521/51). O sistema não garante que essa taxa seja legal. Deseja registrar mesmo assim, ciente disso?"

3. Com a confirmação, grava `acima_limite_legal=true` e `ciencia_aviso_legal_em`. O empréstimo aparece com um selo ⚠️ e o relatório de empréstimos traz uma seção "Condições acima da referência legal".
4. Juros compostos com período menor que um ano geram uma observação de vedação nas `condicoes_texto`. Sugira juros simples.
5. As condições contratuais ficam registradas de forma transparente em `condicoes_texto`: principal, taxa, base, modalidade, períodos, parcelas, vencimentos e taxa equivalente. Daí sai um PDF "Resumo das condições", que pode servir de base para um contrato redigido por um advogado.

2. Compra e venda de medicamentos

O que considerar:

- **Lei 5.991/1973 e Lei 13.021/2014:** só farmácias e drogarias licenciadas, com responsável técnico farmacêutico presente, podem dispensar medicamentos ao consumidor.
- **Lei 6.360/1976:** todo medicamento precisa de registro ou notificação na ANVISA. Vender produto sem registro, falsificado ou de procedência ignorada é crime hediondo (Código Penal, art. 273).
- **AFE (Autorização de Funcionamento da ANVISA)**, licença sanitária local (Vigilância Sanitária municipal ou estadual) e Certidão de Regularidade do CRF.
- **RDC 44/2009** (boas práticas farmacêuticas) e **RDC 471/2021** (antimicrobianos: receita em 2 vias, com retenção).
- **Portaria SVS/MS 344/1998** (controle especial, listas A, B, C...): exige Autorização Especial, notificação de receita, escrituração e transmissão ao **SNGPC**. Itens dessa portaria **não podem ser tratados como mercadoria comum**.
- **Rastreabilidade:** lote e validade em cada entrada e saída. O Sistema Nacional de Controle de Medicamentos (SNCM, Lei 11.903/2009) prevê o código único (IUM); confirme a etapa de implantação em vigor.
- Agonistas de GLP-1 e GIP (semaglutida, tirzepatida e similares) são medicamentos de **venda sob prescrição**, e a ANVISA tem determinado a apreensão de versões sem registro e manipuladas irregulares. Produtos assim devem ser cadastrados com `classe_regulatoria='tarja_vermelha'` ou superior, e com o registro ANVISA preenchido.

Como o sistema aplica isso (função `checar_venda_medicamento` no schema):

Situação	Comportamento
`classe_regulatoria='nao_medicamento'` (mercadoria comum ou correlato)	Venda livre
Qualquer medicamento sem AFE, licença sanitária e responsável técnico em Configurações	**Bloqueia** (`LICENCA_OBRIGATORIA`)
Licença vencida	**Bloqueia** (`LICENCA_VENCIDA`)
Medicamento sem registro ANVISA cadastrado	**Bloqueia** (`REGISTRO_ANVISA_AUSENTE`)
Tarja vermelha sem receita registrada	**Bloqueia** (`RECEITA_OBRIGATORIA`)
Antimicrobiano ou Portaria 344 sem receita retida	**Bloqueia** (`RECEITA_RETENCAO`)
Portaria 344 sem `autorizado_controlados=true`	**Bloqueia** (`CONTROLADO_BLOQUEADO`)
Lote vencido	Nunca é vendido (FEFO ignora o lote)

- No cadastro de produto, a classe regulatória é obrigatória quando o item é medicamento. A IA deve perguntar a classe ao cadastrar um produto que pareça ser medicamento.
- A IA nunca ensina a contornar esses bloqueios. Ela explica o que precisa ser regularizado.
- Cada receita fica guardada (`receitas_medicas`), ligada à venda, com o arquivo da imagem no Storage privado.

3. LGPD (Lei 13.709/2018)

- Os dados de clientes (terceiros) entram com finalidade definida: gestão de vendas, empréstimos e cobrança.
- Colete o mínimo necessário. CPF e endereço são opcionais e ficam cifrados.
- Dados de saúde (receitas) são **dados pessoais sensíveis** (art. 11): acesso restrito, Storage privado e registro do acesso na auditoria.
- Em Configurações → Privacidade: exportar todos os dados (portabilidade), anonimizar um cliente e explicar quais terceiros recebem dados (Anthropic, para interpretar mensagens; Google, para os eventos).
- Incidentes de segurança: o registro na `auditoria` e nos logs ajuda a cumprir o dever de comunicação (art. 48).

4. Salário

O salário da PM é **receita pessoal** (`escopo='pessoal'`, categoria Salário PM). Ele nunca entra no faturamento comercial nem nos relatórios do negócio.

08

Relatórios em PDF

Tela `/relatorios`: escolha o **período** (os mesmos atalhos do dashboard ou datas livres), os **módulos** (caixas de seleção; dá para juntar vários num só PDF) e, quando fizer sentido, o cliente. O botão "Gerar PDF" baixa o arquivo, e "Pré-visualizar" mostra o PDF numa janela.

Relatórios

1. Financeiro geral (todos os indicadores do dashboard + DRE simplificada pessoal e comercial)
2. Empréstimos (carteira, situação, condições, selos ⚠️ de taxa acima da referência)
3. Juros recebidos (por mês e por cliente; previsto × realizado)
4. Vendas (lista, totais, faturamento × lucro, margem)
5. Compras (por fornecedor e produto, pagas e pendentes)
6. Estoque (inventário valorizado, lotes, validades, alertas)
7. Lucro (vendas + juros, por mês)
8. Contas a pagar
9. Contas a receber (com aging: em dia, 1–30, 31–60 e mais de 60 dias de atraso)
10. Despesas pessoais (por categoria e mês)
11. Salarial (contracheques, bruto, líquido, descontos e variação)
12. Comparativo mensal e anual
13. Individual por cliente (ficha + histórico completo + pendências)

Layout (pdfmake)

- A4 retrato (paisagem para tabelas largas), margens de 40 pt e fonte Roboto (embutida no pdfmake).
- **Cabeçalho** em todas as páginas: "GESTOR FINANCEIRO INTELIGENTE", o nome do relatório, o período e a linha fina na cor de destaque.
- **Rodapé:** "Emitido em dd/mm/aaaa hh:mm por " e "Página X de Y".
- **Capa resumida** (para relatórios com vários módulos): cartões de indicadores, desenhados como tabelas de 2×4.
- **Tabelas:** cabeçalho com fundo cinza-claro, linhas zebradas, números alinhados à direita no formato `R$ 1.234,56`, **subtotal** por grupo e **total geral** em negrito, com a linha do cabeçalho repetida em cada página (`headerRows: 1`).
- **Gráficos:** renderize o mesmo componente Recharts escondido na tela, converta o SVG em PNG (canvas, 2× para ficar nítido) e insira com `image`. Pelo menos um gráfico por módulo que tenha série temporal.
- **Notas:** as definições (faturamento ≠ lucro), a ressalva legal no relatório de empréstimos e um aviso de "dados confidenciais (LGPD)".
- Nome do arquivo: `gfi-<relatorio>-<de>_<ate>.pdf`.

Fonte dos dados

Cada relatório chama uma consulta das views (`v_obrigacoes`, `v_vendas_mensal`, `v_clientes_resumo`, `v_saldo_contas`, `v_estoque_alertas`) ou das tabelas, filtrando pelo período e respeitando o escopo. Os totais são **recalculados** a partir das linhas listadas e conferidos contra o resumo: se não baterem, o relatório mostra um aviso em vez de esconder a diferença.

A IA também gera relatórios: "me manda o relatório de vendas de outubro em PDF" → a ferramenta `gerar_relatorio` devolve um link assinado (Storage privado, válido por 24 h) para o PDF gerado por uma Edge Function com a mesma definição.

09

Roteiro de testes

Rode contra o banco real do projeto, com um usuário de teste, e mostre ao usuário o resultado de cada passo. Este roteiro já foi validado no Postgres 16 com o `assets/schema.sql`. Os números esperados abaixo são os obtidos nesse teste.

A. Cálculos

`node --test scripts/calculos.test.mjs` → 11 testes passando (juros simples e composto, taxa equivalente, parcelas, imputação no pagamento, faturamento × lucro, crescimento, datas em português, telefone, títulos de evento).

B. Fluxos no banco (como usuário autenticado)

#	Ação	Esperado
1	`inicializar_usuario()` duas vezes	21 categorias, 2 contas, sem duplicar
2	`upsert_cliente('Valverde','71 99401-4023')`	`criado`
3	`upsert_cliente('Valverdi','(71) 99401-4023')`	`existente` (pelo telefone)
4	`buscar_clientes('994014023')` e `buscar_clientes('Valverd')`	encontra Valverde
5	Compra de 20 TZ a R$ 80, vencimento 20/11	total 1.600; estoque 20; custo 80; margem 46,67%
6	Venda de 1 TZ a R$ 150 para o Valverde, vencimento 14/12, chave `msg-001`	total 150; custo 80; lucro 70; estoque 19
7	Repetir com a mesma chave	devolve `repetida:true`, sem gravar
8	Repetir sem chave	`POSSIVEL_DUPLICIDADE`; estoque continua 19
9	Fila da agenda	2 itens (compra e venda), um por obrigação
10	Pagamento de 150	obrigação e venda `pago`; a fila continua com 1 item para essa obrigação (upsert → marcar PAGO)
11	Empréstimo de R$ 1.000 ao João, 10% a.m., 1 período, sem ciência	`AVISO_LEGAL_PENDENTE` (10% contra 1% de referência)
12	O mesmo com `ciente_aviso_legal:true`	juros 100; total 1.100; `acima_limite_legal`
13	Pagamento parcial de 500	juros 100 + principal 400; saldo 600; `parcial`
14	Lançamentos	saída patrimonial 1.000; entrada operacional 100 (juros); entrada patrimonial 400
15	`alterar_vencimento` +45 dias	evento `prorrogacao`; a fila atualiza o mesmo evento
16	Pagamento de 700	`VALOR_ACIMA_DO_SALDO`
17	Pagamento de 600	empréstimo `quitado`; lucro com juros = 100
18	Despesa de energia de R$ 350, paga	escopo pessoal, categoria Energia
19	Contracheque: 5.200 + 1.300 − 650 − 400	bruto 6.500; descontos 1.050; líquido 5.450
20	Venda de antimicrobiano sem licença	`LICENCA_OBRIGATORIA`
21	Saldos	Carteira pessoal 5.100,00; Caixa do negócio 250,00
22	`v_vendas_mensal`	faturamento 150; lucro 70; margem 46,67; ticket 150; 1 cliente novo
23	`select` na tabela `integracoes`	`permission denied` (tokens protegidos)
24	`delete from vendas`	`EXCLUSAO_BLOQUEADA`
25	Logar como outro usuário	vê 0 clientes, 0 vendas e 0 obrigações (RLS)

C. IA (Edge Function `ia-comando`), com o usuário de teste

Envie os 5 comandos do pedido original, na ordem: compra, venda, pagamento, empréstimo e despesa. Para cada um, confira que: - a resposta tem o bloco de confirmação com os números retornados pelo banco; - `ia_operacoes.ferramentas` mostra as chamadas esperadas (ver ia-central-de-comando.md); - reenviar a mesma mensagem (mesmo `mensagem_id`) não duplica nada.

Casos de borda: "Vendi para o João" (vários Joões → pergunta qual); "Emprestei 500 pro Pedro" (falta a taxa → pergunta); "Pedro pagou 100" (pagamento parcial → informa o saldo); "Vendi TZ" sem estoque → pergunta se quer lançar a compra.

D. Google Agenda (com a conta real do usuário)

Siga a seção 6 de google-agenda.md e anexe ao relatório final os `htmlLink` dos eventos criados e atualizados.

E. Front-end

- Recarregar a página não perde nada; tudo vem do banco.
- Celular (375 px): menu inferior, chat em tela cheia, tabelas viram cartões.
- `npm run build` sem erros; teste de fumaça com Playwright: login → chat → venda → aparece em Vendas e em Contas.

10

Instalação e publicação

Pré-requisitos

- Node.js 20+, Git e a Supabase CLI (`npm i -g supabase`).
- Contas: Supabase (grátis), Anthropic (chave da API em console.anthropic.com), Google Cloud (grátis) e Vercel ou Netlify (grátis).

1. Banco e back-end (Supabase)

```
supabase login
supabase link --project-ref <ref-do-projeto>
supabase db push                      # aplica supabase/migrations/0001_schema.sql
# No painel: Database → Extensions → habilite pg_cron e pg_net
# SQL Editor: cole os comandos cron.schedule do final do schema (troque <PROJETO>)
#             e guarde a service_role_key no Vault: select vault.create_secret('<chave>', 'service_role_key');
supabase secrets set ANTHROPIC_API_KEY=sk-ant-... \
  GOOGLE_CLIENT_ID=... GOOGLE_CLIENT_SECRET=... \
  GOOGLE_REDIRECT_URI=https://<ref>.supabase.co/functions/v1/google-oauth/callback \
  APP_URL=https://<seu-app>.vercel.app TOKEN_ENCRYPTION_KEY=$(openssl rand -base64 32) TZ_PADRAO=America/Bahia
supabase functions deploy ia-comando
supabase functions deploy google-oauth --no-verify-jwt     # o callback recebe o redirecionamento do Google
supabase functions deploy agenda-sync
supabase functions deploy backup
```

Em Authentication → URL Configuration, coloque a `APP_URL` em Site URL e em Redirect URLs.

2. Google Agenda

Siga a seção 1 de `google-agenda.md` (projeto, API, tela de consentimento, credencial web e URI de callback). Depois, no app: Configurações → Agenda → **Conectar Google Agenda** → autorize.

3. Front-end

```
cp .env.example .env     # VITE_SUPABASE_URL e VITE_SUPABASE_PUBLISHABLE_KEY (Settings → API)
npm install
npm run dev              # http://localhost:5173
```

4. Publicação

- **Vercel:** importe o repositório, framework Vite, adicione as duas variáveis `VITE_*` e faça o deploy. Configure o domínio próprio se quiser.
- **PWA:** o `manifest.webmanifest` e o service worker permitem "Adicionar à tela inicial" no celular.
- Atualize `APP_URL` e as URLs do OAuth do Google se o domínio mudar.

5. Primeiro uso

1. Crie a conta (e-mail e senha) e ative o 2FA em Configurações.
2. Confira as contas financeiras (Carteira pessoal e Caixa do negócio) e informe os saldos iniciais.
3. Conecte o Google Agenda.
4. Se for vender medicamentos, cadastre AFE, licença sanitária e responsável técnico em Configurações → Licenças.
5. Revise o limite de referência de juros em Configurações → Conformidade.
6. Abra a Central IA e escreva: "Comprei 20 unidades de TZ por 80 reais cada, pagamento para dia 20."

Custos estimados

Supabase e Vercel: grátis no início. Claude API: centavos por comando (Sonnet). Google Calendar API: grátis.

Backup

Automático e diário (30 versões no Storage). Manual: Configurações → Backup → Exportar (JSON e CSV). Restauração: Configurações → Backup → Restaurar.